

Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences

b-it Bonn-Aachen International Center for Information Technology

Learning Interpretable Scheduling Policies for Online Multi-Robot Task Allocation

via Inductive Logic Programming – PlanRob @ ICAPS 2026

ICAPS 2026, Dublin

M. I. Memon U. Patil I. Awaad Y. M. Youssef

Motivation

Online task allocation in a dynamic warehouse

- Fleet of mobile robots must fulfil a continuous stream of incoming orders
- Runtime decision: which robot handles which order?
- Must account for battery level, proximity to shelf, and current workload
- NP-hard online scheduling – practical systems rely on heuristic or learned policies

Existing approaches	What they cannot do
Heuristics, auctions – assign a robot and output a score or bid	Explain **why** that robot was the right choice. No audit trail.

Problem Formulation

$P_m \mid r_j, \text{eligible} \mid C_{\max}$

- m homogeneous robots as parallel machines
- Orders arrive online with release times r_j – no look-ahead
- Robot **eligible** iff idle and $b_r > b_{\min}$ (threshold 20%)
- Processing time: $p_j = 2\,d(\text{drop-off}, \text{shelf}_j) \,/\, v$
- Objective: minimise makespan $C_{\max}$

Why learn instead of derive?

- NP-hard even offline ($m \geq 2$)
- Online LPT does not model eligibility
- Induce a policy from execution traces

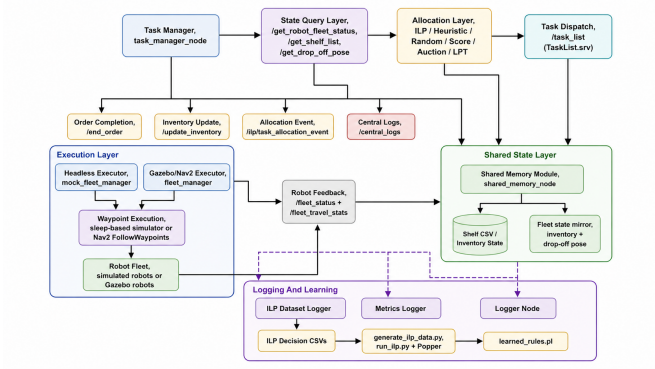
Deferral

No eligible robot $\Rightarrow$ order queued, not force-assigned

System Architecture

Three core ROS 2 nodes + offline learning pipeline

- **Task Manager** – receives orders, queries state, invokes strategy
- **Fleet Manager** – tracks location/battery/availability; Nav2 dispatch
- **Shared Memory** – shelf locations, drop-off pose



Offline pipeline

Logs → Popper ILP → `learned_rules.pl` → Janus bridge → runtime

ILP Learning Pipeline

From execution logs to symbolic Prolog rules

Training data: one row per robot per allocation event
(idle status, workload, battery, distance, dispatch decision) **Symbolic discretisation:**

- Battery → `{low, medium, high}`
- Distance → `{near, medium, far}`
- Workload → `{zero, low, medium, high}`

Teacher (= Heuristic baseline): eligibility → min workload → min distance

Key derived predicates

`suitable_robot(T,R)`
idle + battery $> b_{\min}$

`closest_least_busy_suitable(T,R)`

`no_suitable_robot(T)`

Popper search

Targets: `assigned/2` and `defer/1`
Max 3 rules, 4 variables, 5 literals

The Learned Policy

Two Prolog clauses induced by Popper from labelled allocation examples

```
assigned(T, R) :-
   closest_least_busy_suitable(T, R).
defer(T) :-
   no_suitable_robot(T).
```

Rule 1 in one sentence

Assign to the closest eligible idle robot with the lowest active workload

Scheduling interpretation

Three concepts in strict priority order:

1. Eligibility – idle $+ b_r > b_{\min}$
2. Load balance – min workload tier
3. Proximity – min distance within tier

Rule 2

Deferral induced from data – a learned precondition on action executability

Explanation Generation

Human-readable justification for every dispatch decision

Decision	Fleet State	Explanation
Assign	15 idle robots, all 100%. R2 closest at 4.21 m.	R2 matched `closest_least_busy_suitable`. Cited: high battery, near, zero workload.
Assign + tie-break	R10 and R14 tied at 4.70 m.	Battery tie-break: R10 (100%) over R14 (92.9%). Tie-break reported explicitly.
Defer	All 10 robots busy (workload 1).	All-busy is one instance where `no_suitable_robot(T)` holds. Learned rule `defer(T) :- no_suitable_robot(T)` fires. Task queued.

600 decisions

589 cited Rule 1 directly; 11 applied tie-break; 0 used fallback

Allocator runtime

5.75 ms ($n = 5$) – 33.27 ms ($n = 15$); negligible vs. job execution time

Experimental Setup

Headless warehouse simulation – reproducible without Gazebo

- 6 strategies: Random, Score, Heuristic, Auction, Online LPT, ILP
- 4 fleet sizes: $n \in \{5, 10, 15, 20\}$ – resource-constrained to over-provisioned
- 50 orders at 1 Hz per trial; 3 trials; seeded RNG
- $v = 2.0$ m/s, drain 0.7%/m, recharge 5%/s, $b_{\min} = 20\%$
- All 6 strategies: **100% task success rate**

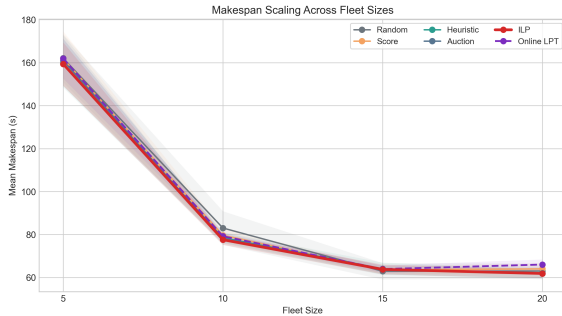
Regimes	
$n = 5$	resource-constrained (hardest)
$n = 10$	moderately provisioned
$n = 15$	lightly loaded
$n = 20$	over-provisioned

Results: Makespan

Mean makespan (s) – lower is better; bold = best per fleet size

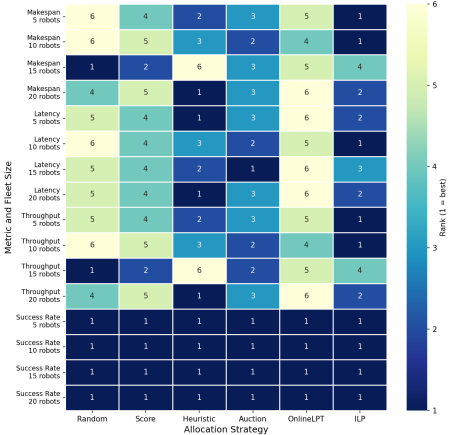
Strategy	n=5	n=10	n=15	n=20
Random	162.1	83.0	**62.9**	63.0
Score	161.8	79.7	63.6	64.0
Heuristic	159.8	78.5	64.1	**61.9**
Auction	160.9	78.2	63.6	62.0
ILP	**159.4**	**77.6**	63.8	61.9
Onl. LPT	161.9	79.2	63.9	66.0

ILP average rank: **2.0** across all fleet sizes (Auction 2.75 · Heuristic 3.0 · Online LPT 5.0)



Results: Overall Ranking

Ranking heatmap – darker = better rank (1 = best)



Key findings

- ILP is empirically competitive; strongest at $n = 5$, $n = 10$ where deferral matters most
- At $n = 20$: all top strategies within 0.17 s – margins are small
- Heuristic is the teacher: ILP's contribution is compressing that behavior into 2 executable symbolic rules
- Uniquely: a human-readable justification for every decision – no baseline provides this

Conclusion

1. **ILP learns the scheduling rule without being given the objective function or algorithm**
   Popper received only labelled traces over a scheduling-relevant predicate vocabulary. It recovered the allocation rule and deferral condition from data alone.
2. **Empirically competitive – best average rank (2.0) over 3 trials per fleet size**
   Strongest under congestion; resource usage (travel, battery) matches the best hand-coded baselines at all fleet sizes.
3. **Human-readable audit trail – absent from all evaluated baselines**
   Every dispatch decision justified by matched rule conditions; no post-hoc interpretation required.

Future work

Heterogeneous fleets · Stochastic arrivals · Online retraining · Larger fleet scaling

Limitations

Backup – for Q&A

- Only 3 trials per fleet size – trends are consistent but no confidence intervals reported
- Homogeneous fleet – all robots identical speed, payload, battery dynamics
- Fixed 1 Hz arrival rate – Poisson arrivals not evaluated (deferral rule is stateless, generalization expected)
- Allocator runtime grows with fleet size: 33 ms at $n = 15$; Prolog assert/retract may need optimization at scale
- Online retraining pipeline implemented but not evaluated
- Atomic orders only – task decomposition assumed upstream

Secondary Metrics

Backup – for Q&A

Strategy	Runtime (ms)	Travel (m)	Battery (%)
Random	0.097	23.85	16.70
Score	0.101	22.53	15.77
Heuristic	0.095	21.96	15.37
Auction	0.098	22.02	15.41
ILP	16.257	21.98	15.38
Online LPT	0.087	24.02	16.81

Table 1: ILP travel and battery match Heuristic/Auction. Runtime is $\sim 160\times$ higher but negligible vs. job execution time (seconds).

ILP deferral rate: 58.2% at $n = 5 \rightarrow$ 31.8% at $n = 10 \rightarrow$ 0.7% at $n = 15 \rightarrow$ 0% at $n = 20$